

L3: Case-sensitive, Quotes, Insert data

RECAP from Lab #2

- Import / Export data
- Query

```
LOAD DATA INFILE '/path/to/books.csv'  
INTO TABLE Books  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS  
(book_id, title, author, publication_year,  
genre, price);
```

```
SELECT title, author  
FROM Books  
WHERE price > 10;
```

```
SELECT author, COUNT(*) AS  
book_count  
FROM Books  
GROUP BY author  
HAVING COUNT(*) > 1;
```

3.1 MySQL Case Sensitivity Guide

Types	Case-Sensitive?
Database / Table Names	• Windows: Case-insensitive • Linux/Unix: Case-sensitive • macOS: Case-insensitive by default
Column Names	Case-insensitive by default
String Comparisons	Case-sensitivity depends on collation Common collations: • _ci suffix: Case-insensitive (e.g., utf8mb4_general_ci) • _cs suffix: Case-sensitive (e.g., utf8mb4_bin) • _as suffix: Accent-sensitive

3.2 SQL Quotes Usage Guide

Quote Type	Syntax	Key Features	Common Uses	Examples
Single Quote (')	'text'	- Standard for string literals - Must escape single quotes with another single quote - Most commonly used in MySQL	- String values - Text fields - Date/time literals - Default values	<pre>SELECT * FROM users WHERE name = 'John' 'It's working' '2024-01-28'</pre>
Double Quote (")	"text"	- Only works if ANSI_QUOTES SQL mode is enabled - Less commonly used for strings - Primary use is for identifiers in ANSI mode	- Identifier quoting when configured - Database/table/column names in ANSI mode	<pre>SELECT * FROM "table_name" CREATE TABLE "my-table" SELECT "column_name" FROM users</pre>
Backtick (`)	`text`	- Used for identifiers (databases, tables, columns) - Never needed for string literals - MySQL-specific syntax	- Database names - Table names - Column names - Especially when names contain spaces or reserved words	<pre>SELECT `first name` FROM `users` CREATE TABLE `my-table` SELECT `order` FROM `sales`</pre>

When to Use and Not Use Quotes

Scenario	With Quotes	Without Quotes	Explanation
String Values	<code>name = 'John'</code>	<code>amount = 100</code>	- Quotes needed for text/strings - No quotes for numbers
Date/Time	<code>date = '2024-01-28'</code>	<code>CURRENT_DATE</code>	- Quotes for date literals - No quotes for date functions
Boolean	<code>status = 'true'</code>	<code>is_active = TRUE</code>	- Some DBs use quotes for boolean strings - MySQL accepts boolean without quotes
NULL	<code>middle_name = 'NULL'</code>	<code>middle_name IS NULL</code>	- Quoted NULL means literal string "NULL" - Unquoted NULL means null value
Column Names	<code>`first name`</code>	<code>first_name</code>	- Quotes needed if contains spaces/special chars - No quotes needed for simple names
Table Names	<code>`my-table`</code>	<code>employees</code>	- Quotes needed if contains hyphens/spaces - No quotes for standard names

When to Use and Not Use Quotes

Scenario	With Quotes	Without Quotes	Explanation
Numbers	<code>id = '123'</code>	<code>id = 123</code>	- Numbers don't need quotes - Adding quotes converts to string
Functions	<code>name = 'UPPER'</code>	<code>UPPER(name)</code>	- Quotes for literal string - No quotes for function calls
Aliases	<code>AS 'My Count'</code>	<code>AS count</code>	- Quotes needed if alias has spaces - No quotes for simple aliases
Reserved Words	<code>`order`</code>	<code>customer_order</code>	- Quotes needed for reserved words - No quotes if not reserved

Suggested Quotes Practices

1. Only use quotes when necessary
2. Avoid naming objects that require quotes
3. Be consistent with quote style
(backticks for identifiers, single quotes for strings)
4. Always quote string and date literals
5. Never quote numbers unless specifically needed as strings
6. Use proper NULL syntax without quotes
7. Quote column/table names only when they:
 - Contain spaces
 - Use special characters
 - Are reserved words
 - Are case-sensitive

3.3 INSERT Statement

Syntax

```
INSERT INTO table_name(column1, column2,...)
VALUES (value1, value2,...);
```

Example

```
INSERT INTO tasks(title, priority)
VALUES ('Learn MySQL INSERT Statement', 1);
```

task_id	title	start_date	due_date	priority	description
NULL	NULL	NULL	NULL	NULL	NULL

↓

task_id	title	start_date	due_date	priority	description
1	Learn MySQL INSERT Stat...	NULL	NULL	1	NULL
NULL	NULL	NULL	NULL	NULL	NULL

INSERT Statement

Example

```
INSERT INTO tasks(title, priority)
VALUES
    ('My first task', 1),
    ('It is the second task', 2),
    ('This is the third task of the week', 3);
```

task_id	title	start_date	due_date	priority	description
1	Learn MySQL INSERT Stat...	NULL	NULL	1	NULL
2	My first task	NULL	NULL	1	NULL
3	It is the second task	NULL	NULL	2	NULL
4	This is the third task of th...	NULL	NULL	3	NULL

3.4 Insert a table into another table

Syntax

```
INSERT INTO table_name(column1, column2,...)
SELECT <statement>;
```

Example

```
INSERT INTO suppliers (
    supplierName, phone, addressLine1, addressLine2, city,
    state, postalCode, country, customerNumber
)
SELECT
    customerName, phone, addressLine1, addressLine2, city,
    state, postalCode, country, customerNumber
FROM
    customers
WHERE
    country = 'USA' AND
    state = 'CA';
```

Table: Customers

customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1	addressLine2	city	state	postalCode	country	sale
124	Mini Gifts Distributors Ltd.	Nelson	Susan	4155551450	5677 Strong St.	NULL	San Rafael	CA	97562	USA	1161
129	Mini Wheels Co.	Murphy	Julie	6505555787	5557 North Pendale Street	NULL	San Francisco	CA	94217	USA	1161
161	Technics Stores Inc.	Hashimoto	Juri	6505556809	9408 Furth Circle	NULL	Burlingame	CA	94217	USA	1161
205	Toys4GrownUps.com	Young	Julie	6265557265	78934 Hillside Dr.	NULL	Pasadena	CA	90003	USA	1161
219	Boards & Toys Co.	Young	Mary	3105552373	4097 Douglas Av.	NULL	Glendale	CA	92561	USA	1161



Table: Suppliers

supplierNumber	supplierName	phone	addressLine1	addressLine2	city	state	postalCode	country	customerNumber
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL



Table: Suppliers

supplierNumber	supplierName	phone	addressLine1	addressLine2	city	state	postalCode	country	customerNumber
1	Mini Gifts Distributors Ltd.	4155551450	5677 Strong St.	NULL	San Rafael	CA	97562	USA	124
2	Mini Wheels Co.	6505555787	5557 North Pendale Street	NULL	San Francisco	CA	94217	USA	129
3	Technics Stores Inc.	6505556809	9408 Furth Circle	NULL	Burlingame	CA	94217	USA	161
4	Toys4GrownUps.com	6265557265	78934 Hillside Dr.	NULL	Pasadena	CA	90003	USA	205
5	Boards & Toys Co.	3105552373	4097 Douglas Av.	NULL	Glendale	CA	92561	USA	219
6	Collectable Mini Designs Co.	7605558146	361 Furth Circle	NULL	San Diego	CA	91217	USA	239
7	Corporate Gift Ideas Co.	6505551386	7734 Strong St.	NULL	San Francisco	CA	94217	USA	321
8	Men 'R' US Retailers, Ltd.	2155554369	6047 Douglas Av.	NULL	Los Angeles	CA	91003	USA	347
9	The Sharp Gifts Warehouse	4085553659	3086 Ingle Ln.	NULL	San Jose	CA	94217	USA	450
10	West Coast Collectables Co.	3105553722	3675 Furth Circle	NULL	Burbank	CA	94019	USA	475
11	Signal Collectibles Ltd.	4155554312	2793 Furth Circle	NULL	Brisbane	CA	94217	USA	487
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3.5 Query into another table

Syntax

```
INSERT INTO table_name(column1, column2, ...)
VALUES (
    (SELECT <statement1>),
    (SELECT <statement2>)
);
```

Example

```
INSERT INTO stats(totalProduct, totalCustomer, totalOrder)
VALUES (
    (SELECT COUNT(*) FROM products),
    (SELECT COUNT(*) FROM customers),
    (SELECT COUNT(*) FROM orders)
);
```

Table: products

COUNT(*)

110

Table: customers

COUNT(*)

122

Table: orders

COUNT(*)

326



Table: stats

totalProduct

totalCustomer

totalOrder



Table: stats

totalProduct

totalCustomer

totalOrder

110

122

326

3.6 INSERT ON DUPLICATE KEY UPDATE statement

Syntax

```
INSERT INTO table_name (column1, column2, ...)
VALUES (...)
AS new
ON DUPLICATE KEY UPDATE
    column1 = new.column1,
    column2 = new.column2 + 1;
```

- If a duplicate key violation occurs, you can use the INSERT ON DUPLICATE KEY UPDATE statement to **update** existing rows instead of throwing an error.
- This INSERT ON DUPLICATE KEY UPDATE statement is useful when you **deal with unique constraints or primary keys**.

id	name	age	salary	bonus
1	Jane Doe	25	120000.00	0.00



```
INSERT INTO employees2(id, name, age, salary)
VALUES(1, 'Jane Doe', 26, 130000)
```



Error Code: 1062. Duplicate entry '1' for key 'employees2.PRIMARY'

```
INSERT INTO employees2(id, name, age,
salary)VALUES(1, 'Jane Doe', 26, 130000)
AS new
ON DUPLICATE KEY UPDATE
age = new.age,
salary = new.salary;
```



id	name	age	salary	bonus
1	Jane Doe	26	130000.00	0.00

3.7 Date and Time

DATE data-type Format

'YYYY-MM-DD'

Inserting the current date

CURRENT_DATE

Inserting the UTC date

UTC_DATE()

Converting from String to Date

STR_TO_DATE(...)

Example

```
INSERT INTO events(event_name, event_date)
VALUES
('Job # 1', '2024-1-01'),
('Job # 2', CURRENT_DATE),
('Job # 3', UTC_DATE()),
('Job # 4', STR_TO_DATE('22/04/2025', '%d/%m/%Y'))
;
```

id	event_name	event_date
1	Job # 1	2024-01-01
2	Job # 2	2024-12-06
3	Job # 3	2024-12-06
4	Job # 4	2025-04-22
NULL	NULL	NULL

Date and Time

DATETIME data-type format

```
'YYYY-MM-DD HH:MM:SS'
```

Example

Inserting the current datetime

```
NOW()
```

Converting from String to datetime

```
STR_TO_DATE(...)
```

```
INSERT INTO events(event_name, event_datetime)
VALUES
('Job # 1', '2024-9-28 19:30:35'),
('Job # 2', NOW()),
('Job # 3', str_to_date('10/28/2024 20:15:30',
                        '%m/%d/%Y %H:%i:%s')
);
```

id	event_name	event_datetime
1	Job # 1	2024-09-28 19:30:35
2	Job # 2	2024-12-06 14:41:52
3	Job # 3	2024-10-28 20:15:30